DQN and Rainbow: A Comparative Study

Sergio Herreros Pérez [1] **Adrián López-Lanchares Echezarreta** [1] **Ignacio Bayón Jiménez-Ugarte** [1]

Abstract

Learning continuous control from high-dimensional pixels is a major challenge in Deep Reinforcement Learning. This paper evaluates classic value-based algorithms on pixel-based continuous control tasks using the DeepMind Control Suite. We compare a Branching Deep Q-Network (DQN) against the Rainbow DQN algorithm across environments ranging from the single-legged Hopper to the highly unstable bipedal Humanoid. Through custom reward shaping, action discretization, and distributional calibration, we demonstrate that value-based methods can learn stable walking directly from raw pixels. Ultimately, while standard DQN fails on complex bipedal tasks, the combined algorithmic improvements of Rainbow enable successful and stable high-dimensional continuous control. Code available at https://github.com/IgnacioBayon/drl-zoo

1. Introduction

Deep Reinforcement Learning (DRL) has achieved remarkable success, yet learning continuous control policies directly from visual inputs remains computationally demanding (Mnih et al., 2015). Traditional actor-critic methods, such as Soft Actor-Critic (SAC) (Haarnoja et al., 2018), dominate these tasks due to their natural handling of continuous actions. However, value-based methods offer compelling off-policy sample efficiency. Recent advances in action-space discretization and network branching have made it possible to adapt Q-learning for continuous domains (Tavakoli et al., 2019), providing a viable alternative to standard actor-critic setups.

To benchmark these capabilities, the DeepMind Control Suite (Tassa et al., 2018) provides a rigorous physics-based simulation framework. In this study, we focus on learning movement strictly from visual pixels, utilizing environments that span from the moderately complex Hopper to the highly unstable 17-joint Humanoid.

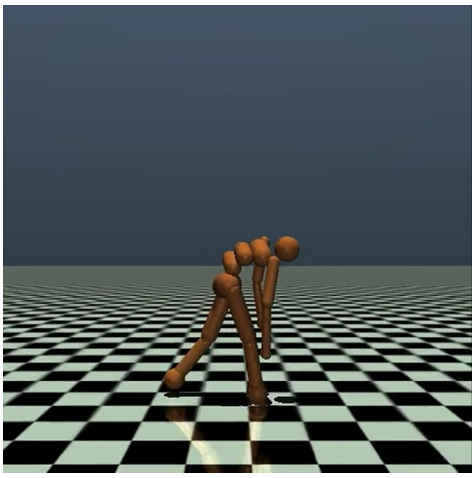

Figure 1. Example of the learned Humanoid walking posture using Rainbow, depicting the folded torso and stable gait.

This paper systematically compares a baseline Deep Q-Network (DQN) against the heavily augmented Rainbow DQN algorithm (Hessel et al., 2018) on these high-dimensional tasks. We investigate how enhancements like distributional value modeling, Prioritized Experience Replay, and multi-step returns enable an agent to map raw pixel stacks to stable joint torques. By evaluating these algorithms across varying difficulty levels, we isolate the components strictly necessary to achieve complex bipedal balance using purely visual observations.

2. Deep Q-Network (DQN)

2.1. Algorithm Overview

Deep Q-Networks (DQN) (Mnih et al., 2015) extend Q-learning by approximating the action-value function $Q(s, a)$ with a neural network. The objective is to learn a policy that maximizes the expected discounted return by minimizing the temporal-difference (TD) error derived from the Bellman optimality equation:

$$Q^*(s, a) = \mathbb{E}\left[r + \gamma \max_{a'} Q^*(s', a')\right]. \tag{1}$$

[1]Master in Artificial Intelligence, Universidad Pontificia Comillas (ICAI). Correspondence to: Sergio Herreros Pérez <sergio.herreros@alu.comillas.edu>, Adrián López-Lanchares Echezarreta <adrianlanchares@alu.icai.comillas.edu>, Ignacio Bayón Jiménez-Ugarte <ibayon@alu.comillas.edu>.

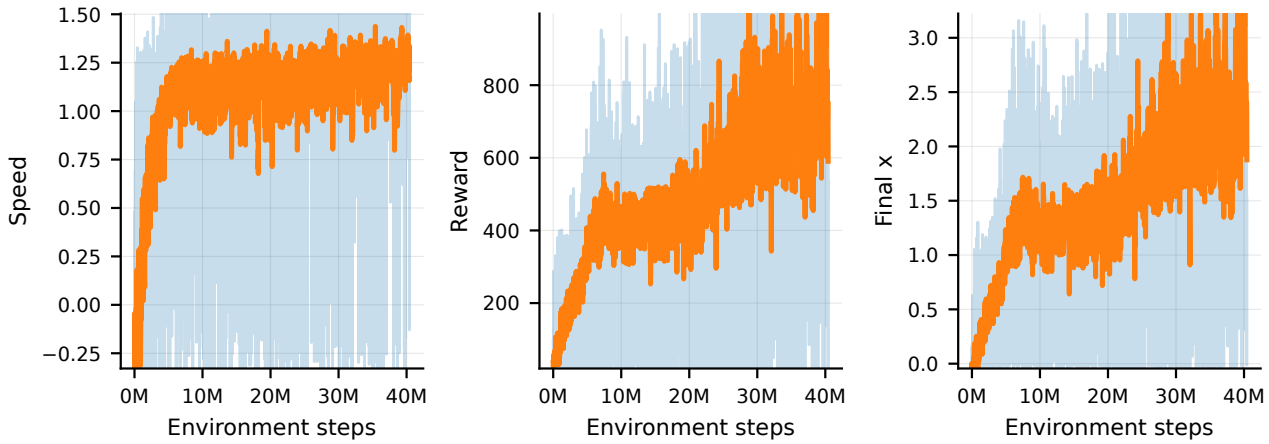

Figure 2. Training metrics of DQN for the Hopper environment (smoothed with an exponential moving average).

DQN optimizes the squared TD loss:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[(Q_\theta(s,a) - y)^2\right], \tag{2}$$

where the target is defined as:

$$y = r + \gamma \max_{a'} Q_{\theta^-}(s', a'). \tag{3}$$

Here, θ^- are the parameters of a target network, updated periodically to stabilize training.

To improve data efficiency and reduce temporal correlations, transitions are stored in a replay buffer and sampled uniformly during training. Exploration is handled via an ϵ-greedy policy, where random actions are selected with probability ϵ, which is annealed over time.

2.2. Handling Discretized Continuous Control

Applying DQN to continuous control requires discretizing the action space. In standard DQN, this discretization implies a combinatorial explosion of K^N possible actions for an environment with N independent joints and K bins per joint.

To bypass this exponential growth, we employ a Branching Dueling Q-Network (BDQN) (Tavakoli et al., 2019). BDQN uses a shared state representation that branches into N independent action-value estimators, reducing network outputs from K^N to $K \times N$. This implicitly decouples joint actuation and it drastically saves parameters. Empirical tests on the Hopper environment showed BDQN achieves similar results to standard combinatorial DQN. Given its superior parameter efficiency, we adopt BDQN as our default architecture to scale effectively to more complex environments.

Additionally, default discretization wrappers map continuous actions to bin centers. We observed that at low resolutions (e.g., $K = 3$), this artificially restricts the agent's torque range, limiting Hopper actions to approximately $[-0.6, 0.6]$ rather than the environment's full $[-1, 1]$ bounds, which severely impedes learning. To solve this, we use a custom discretization that maps to interval extremes, ensuring absolute maximum and minimum forces remain accessible.

2.3. Custom Reward Shaping

Standard implementations of continuous control environments often rely on a linear combination of proportional forward velocity, a constant healthy bonus, and a control penalty based on the absolute magnitude of the current action. To promote smoother gaits and more stable learning from high-dimensional pixel inputs, we heavily modified the reward function.

First, rather than incentivizing unbounded forward acceleration, we encourage the agent to track a specific target velocity. This is achieved using an exponential kernel that smoothly penalizes deviations from the target speed:

$$r_{\text{vel}} = w_{\text{vel}} \exp\left(-(v_x - v_{\text{target}})^2\right), \tag{4}$$

where v_x is the current forward velocity, v_{target} is the desired speed, and w_{vel} is a forward weight. Second, we replace the standard absolute-value control penalty with a temporal smoothing cost. To actively penalize erratic, jittery movements, the control cost is computed as the squared difference between consecutive actions:

$$r_{\text{ctrl}} = -w_{\text{ctrl}} \sum_i (a_{t,i} - a_{t-1,i})^2, \tag{5}$$

with a penalty weight w_{ctrl}. The healthy survival reward is retained with a constant value w_{healthy}.

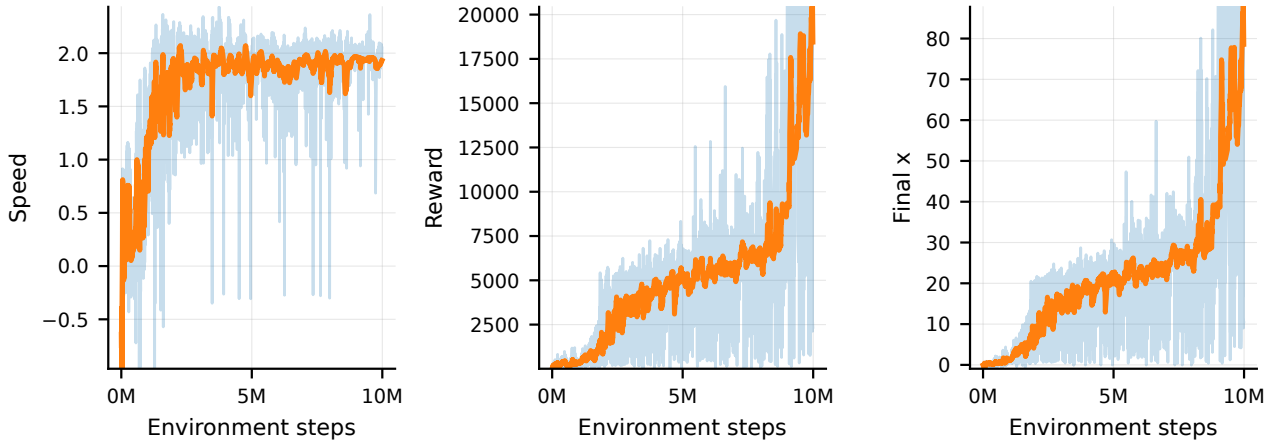

Figure 3. Learning dynamics of Rainbow on the Hopper environment. The agent rapidly converges to a stable, high-speed gait in approximately 10M steps.

2.4. Hopper: Learning from Pixels

Before scaling to more complex challenges, we validate our algorithm on a simpler continuous control task: the Hopper environment. The primary objective is to verify that the agent can learn stable walking relying entirely on visual inputs rather than standard low-dimensional state vectors.

We found the following configurations to be crucial for successful training:

Environment Configuration: The agent operates directly on pixel data, observing 64×64 images with a frame stack of 4 to capture temporal dynamics. We deploy 30 parallel environments to increase data throughput and diversify the replay buffer. Notably, we widen the `healthy_angle_range` to $[-0.4, 0.4]$ (from the default $[-0.2, 0.2]$), allowing the agent to explore more diverse states during the early stages of training.

Reward Configuration: The reward function is shaped to encourage a target velocity of 1.5 units/s, with a velocity reward weight of $w_{\text{vel}} = 1.0$ and weight of $w_{\text{vel}} = 3.0$. The control penalty weight is set to $w_{\text{ctrl}} = 0.001$ to promote smooth actions, while the healthy survival bonus is maintained at $w_{\text{healthy}} = 1.0$.

Algorithm Configuration: We train the Branching DQN (BDQN) for $50,000,000$ environment frames. The action space is mapped to 3 discrete bins representing the extremes of the joint torque intervals. To handle the instability and large Temporal-Difference (TD) errors common when training Q-networks from pixels, we utilize a Huber loss function rather than standard MSE. Exploration follows an ϵ-greedy policy, annealed linearly over a prolonged period of $5,000,000$ frames. Optimization is performed using a learning rate of 10^{-4}, with the target network updated every $10,000$ frames to provide stable bootstrapping targets.

The learning dynamics of this configuration are illustrated in Figure 2. As training progresses, the agent's step-wise speed smoothly converges toward the target velocity of 1.5. Note that it converges to 1.25, which indicates that for the strategy it learned that is the most stable velocity. A higher target may have learned a faster strategy or just collapsed. This convergence is accompanied by a steady increase in both the episodic reward and the final X position, indicating that the combination of our custom reward shaping and the BDQN architecture successfully enables the agent tomaintain balance and move forward relying strictly on high-dimensional pixel observations.

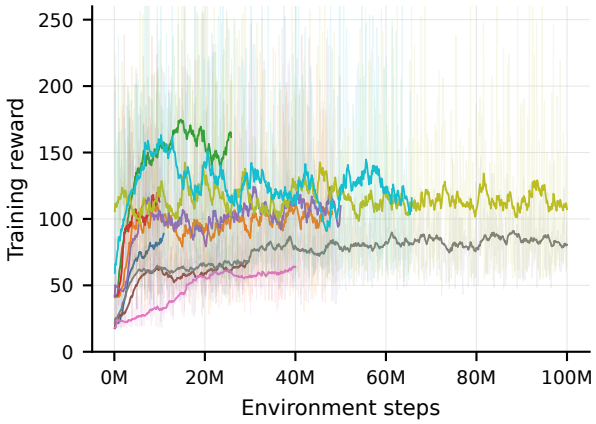

Figure 4. Training reward across multiple hyperparameter sweeps on the Humanoid environment using DQN.

2.5. Humanoid: The Limits of Discretized DQN

Following the successful application of Branching DQN on the Hopper environment, we attempted to scale the algorithm to the highly complex Humanoid environment. This

presents a massive leap in difficulty due to its inherent bipedal instability and a vast continuous action space comprising 17 independent joints.

Environment Tweaks: To mitigate this complexity, we artificially constrained the morphology by freezing several non-essential actuators. Specifically, we locked the Z-axis rotations of the abdomen and hips, and completely disabled all six arm joints (rendering the arms physically limp). By freezing these 9 specific joints ($[1, 4, 8, 11, 12, 13, 14, 15, 16]$), we force the agent to focus purely on the core mechanics of leg actuation and balance. Additionally, to capture the finer nuances of this complex body in the visual state, we slightly increased the observation resolution from 64×64 to 96×96 pixels.

Hyperparameter Sweeps: To find a viable learning configuration, we conducted an exhaustive hyperparameter search. We massively scaled the training horizon (up to 100M frames) and prolonged the ϵ-greedy exploration phase (annealing over up to 50M frames). We tuned replay buffer capacities (up to 500k), learning rates (10^{-5} to 10^{-4}), batch sizes, and target network update frequencies. Furthermore, we experimented heavily with the reward shaping parameters, tweaking target velocities, contact costs, and survival bonuses to guide the agent.

Despite these extensive morphological simplifications and exhaustive hyperparameter sweeps, none of the trials successfully learned to stably walk (see Figure 4). These negative results strongly indicate the fundamental limitations of Q-learning and action discretization—even when mitigated by branching architectures—for high-dimensional continuous control tasks where joint actuations are deeply correlated and highly sensitive to approximation errors.

3. Rainbow DQN

3.1. Algorithm Overview

Rainbow (Hessel et al., 2018) combines several independent extensions of the standard DQN into a single, unified algorithm, drastically improving sample efficiency and training stability. We implement all six core Rainbow components: Double Q-learning, Prioritized Experience Replay (PER), Dueling Networks, Multi-step Returns, Distributional Reinforcement Learning (C51), and Noisy Networks.

Double Q-learning mitigates the overestimation bias inherent to standard Q-learning by decoupling action selection (handled by the online network) from action evaluation (handled by the target network). PER improves data efficiency by prioritizing the sampling of transitions with high temporal-difference errors, ensuring the agent learns from its most surprising experiences. Multi-step returns bootstrap the Q-values over n forward steps, accelerating the propagation of delayed rewards. The dueling architecture splits the neural network into two separate streams (one estimating the state-value and another estimating the state-dependent action advantages) allowing the agent to better evaluate states where specific action choices have minimal impact. Furthermore, distributional RL (C51) models the entire probability distribution of expected returns rather than just their scalar mean, providing a much richer learning signal. Finally, Noisy Networks replace traditional ϵ-greedy exploration by injecting parameterized noise directly into the linear layers of the network, driving state-dependent and mathematically consistent exploration.

3.2. Architecture and Distributional Calibration

Applying Rainbow to high-dimensional, pixel-based continuous control requires careful architectural scaling and distributional calibration. In the C51 formulation, the value distribution is modeled over a discrete categorical support bounded by $[v_{\min}, v_{\max}]$. It is critical to properly calibrate this interval to encompass the true range of possible discounted returns. Given our custom reward shaping with a forward weight of $w_{\text{vel}} = 3.0$ and a healthy bonus of $w_{\text{healthy}} = 1.0$, the maximum single-step reward is approximately $r_{\max} \approx 4.0$.

For an infinite-horizon discounted MDP, the theoretical maximum return is defined as:

$$R_{\max} \approx \frac{r_{\max}}{1 - \gamma} \tag{6}$$

Using our discount factor of $\gamma = 0.99$, we calculate $R_{\max} \approx 400$. Consequently, we calibrated our support bounds to $v_{\min} = -10.0$ and $v_{\max} = 400.0$ distributed across 51 atoms. Poor calibration here severely cripples learning due to distribution clipping.

Additionally, we found that standard network capacities were insufficient for evaluating pixel-based states in this regime. Increasing the hidden dimension of the dueling head from 256 to 512 neurons proved essential; the smaller configuration failed to accurately capture the complex advantage function, whereas 512 neurons provided the necessary representational capacity. To maintain stability and prevent catastrophic updates, we applied a gradient clipping norm of 10.0. Note that unlike our standard DQN approach, we do not employ branching architectures here, as extending branching to the Rainbow algorithm yielded no measurable gains for our setup.

3.3. Hopper: A Leap in Performance

Before tackling the Humanoid, we evaluated our Rainbow configuration on the simpler Hopper environment. The improvements over the baseline BDQN were massive. While standard DQN required approximately 40 million frames to

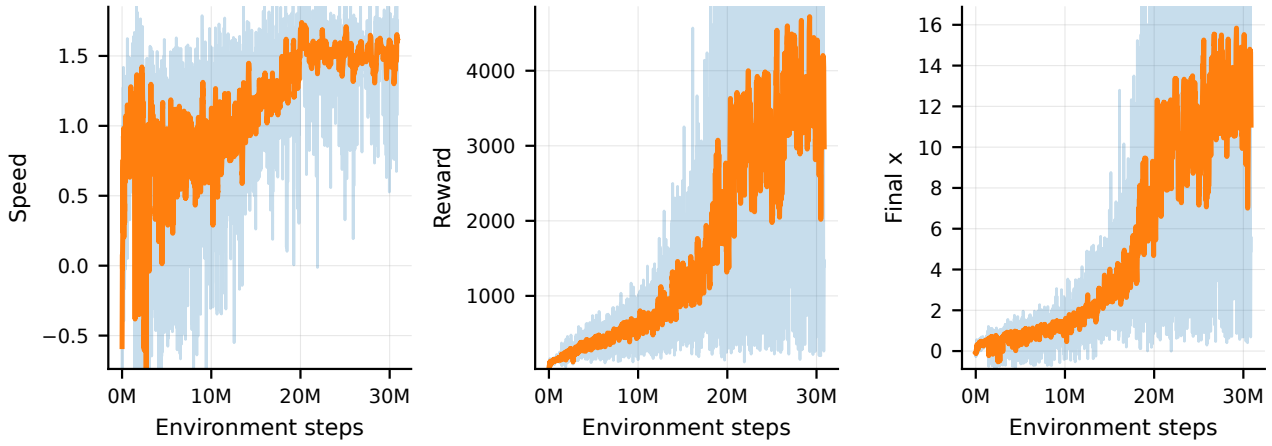

Figure 5. Learning dynamics of Rainbow on the Humanoid-v5 environment. The three subplots illustrate the steady convergence of reward, velocity, and forward distance.

learn a stable gait, Rainbow achieved highly stable walking in just 10 million steps.

Interestingly, Rainbow also discovered a vastly superior behavioral policy. Previously, the standard DQN converged to a sub-optimal speed of 1.25 units/s against a target of 1.5, learning a somewhat fumbling, conservative hop to maintain balance. Rainbow, however, learned to confidently exceed the target, stabilizing at an average velocity of 1.75 units/s. This behavior suggests that learning a rapid, fluid, and continuous motion is dynamically more stable (relying on forward inertia) than the slower, stuttering hops learned by DQN that constantly require the agent to catch its balance. The Rainbow Hopper sustains this velocity indefinitely, traveling entirely out of the standard rendering zone without collapsing.

3.4. Humanoid: Learning Stable Walking Behaviour

Building on the success in the Hopper environment, we applied Rainbow to the heavily constrained Humanoid-v5 task. The environment utilized a 96×96 pixel observation with a frame stack of 4, running across 30 parallel environments. The action space was discretized into 3 bins per joint, mapped to the absolute extremes.

Algorithm Configuration: We trained the model for 50,000,000 frames using a replay buffer of 300,000 transitions and a batch size of 64. We utilized 3-step returns ($n = 3$). Prioritized Experience Replay was configured with an $\alpha = 0.6$, and importance sampling weights (β) were annealed from 0.4 to 1.0 over the first 1,000,000 frames. Exploration was driven dynamically via Noisy Networks initialized with $\sigma_0 = 0.5$. The optimization used a learning rate of 10^{-4} with target network updates occurring every 10,000 frames. Reward parameters were strictly defined: target velocity at 1.0, $w_{\text{vel}} = 3.0$, $w_{\text{healthy}} = 1.0$, and a control penalty of $w_{\text{ctrl}} = 0.01$.

Results and Discussion: Unlike the standard DQN, which completely failed to learn meaningful movement despite exhaustive hyperparameter sweeps, Rainbow successfully achieved stable walking in just 30 million environment steps. Remarkably, it conquered the high-dimensional Humanoid task in fewer steps than the baseline DQN required for the simple Hopper.

As shown in Figure 1, the resulting movement is highly stable but visually unnatural. To maximize the reward function while avoiding catastrophic falls, the agent adopts a posture with its torso folded heavily forward. It effectively drags one leg while using the other for primary propulsion, resembling a crutch-like motion. While this motion is physically awkward, it exploits the physics engine brilliantly, keeping the center of mass extremely low and completely bypassing the bipedal instability.

4. Conclusion

In this work, we explored the viability of classic value-based Reinforcement Learning algorithms for high-dimensional, pixel-based continuous control. By systematically evaluating Branching DQN and Rainbow DQN across the DeepMind Control Suite, we demonstrated that standard Q-learning can successfully navigate complex walking tasks when paired with the right architectural and environmental adaptations.

A primary takeaway from our experiments is the absolute necessity of careful reward shaping. Standard linear velocity rewards often fail because they inadvertently encourage agents to maximize short-term gains through erratic, kamikaze-style forward lunges that result in immediate col-

lapse. By replacing these with a smooth, exponential kernel-based reward, we forced the agent to learn stable and sustainable gaits. Additionally, meticulously tuning the ratio between forward propulsion incentives and healthy survival bonuses proved critical to balancing speed with mechanical stability.

Our results also highlight that successfully learning from raw pixels requires deeply tailored network architectures. The chosen observation resolution and the specific Convolutional Neural Network used for feature extraction must be capable of capturing the fine biomechanical details of the environment. Furthermore, standard network capacities often bottleneck performance in visual regimes. We found that significantly expanding the hidden dimensions of the network (such as upgrading the dueling head to 512 neurons) was strictly essential. Without this expanded capacity, the model cannot properly map extracted latent visual features to accurate action-value estimates.

Finally, applying value-based methods to physical environments demands an effective strategy for action space discretization. As the number of controllable joints increases, agents face a paralyzing combinatorial explosion of possible actions. Our findings reinforce that branching architectures are a highly effective solution to this problem. By independently estimating values for each joint, branching entirely bypasses the exponential growth of the action space, allowing value-based methods to scale gracefully to complex, multi-joint morphologies.

Ultimately, while policy gradient methods remain the default for continuous control, our research confirms that heavily augmented value-based algorithms like Rainbow DQN offer a highly sample-efficient and powerful alternative for visual-based walking tasks.

References

Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pp. 1861–1870, 2018.

Hessel, M., Modayil, J., van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M., and Silver, D. Rainbow: Combining improvements in deep reinforcement learning. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence (AAAI)*, pp. 3215–3222, 2018.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., and Hassabis, D. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533, 2015.

Tassa, Y., Doron, Y., Muldal, A., Erez, T., Li, Y., de Las Casas, D., Budden, D., Abdolmaleki, A., Merel, J., Lefrancq, A., et al. Deepmind control suite. *arXiv preprint arXiv:1801.00690*, 2018.

Tavakoli, A., van Hasselt, H., and Silver, D. Action branching architectures for deep reinforcement learning. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pp. 941–949, 2019.